

15

ARM Symbolic Debug Table Format

This section specifies the format of symbolic debugging data generated by ARM compilers, which is used by the ARM symbolic debugger to support high-level language-oriented, interactive debugging.

15.1	Overview of ARM Symbolic Debug Table Format	15-2
15.2	Order of Debugging Data	15-3
15.3	Representation of Data Types	15-4
15.4	Section Items	15-7
15.5	Procedure Items	15-9

15.1 Overview of ARM Symbolic Debug Table Format

For each separate compilation unit (called a *section*) the compiler produces debugging data, and a special *area* in the object code (see **Chapter 14, ARM Object Format** for an explanation of ARM Object Format, including areas and their attributes). Debugging data are position-independent, containing only relative references to other debugging data within the same section, and relocatable references to other compiler-generated areas.

Debugging data areas are combined by the ARM linker into a single contiguous section of a program image. For details of the ARM linker's capabilities see **Chapter 3, Linker**. For a description of the linker's principal output format see **Chapter 12, ARM Image Format**.

The format of debugging data allows for a variable amount of detail. This potentially allows you to trade off between memory used, disk space used, execution time, and debugging detail.

Assembly-language-level debugging is also supported, though in this case the debugging tables are generated by the linker. If required, the assembler can generate debugging table entries relating code addresses to source lines. Low-level debugging tables appear in an extra section item, as if generated by an independent compilation.

Low-level and high-level debugging are orthogonal facilities, though the debugger allows you to move smoothly between levels if both sets of debugging data are present in an image.

15.1.1 Terminology

The terms *byte*, *word*, and *halfword* are used to mean:

- byte 8 bits, usually considered unsigned
 - word 32 bits (4 bytes), often considered signed
 - halfword 16 bits (2 bytes), also called a *short*
- Halfwords are unused, except in the long form of *LineInfo* items.



15.2 Order of Debugging Data

A debug data area consists of a series of *items*. The arrangement of these items mimics the structure of the high-level language program itself.

For each debug area, the first item is a *section* item, giving global information about the compilation, including a code identifying the language, and flags indicating the amount of detail included in the debugging tables.

Each definition datum, function, procedure, etc., in the source program has a corresponding debug data item, which appear in an order corresponding to the order of definitions in the source. This means that nested structures in the source program are preserved in the debugging data, and the debugger can use these structure to make deductions about the scope of various source-level objects. For procedure definitions, there are two debug items:

<i>procedure</i> item	marks the definition itself
<i>endproc</i> item	marks the end of the procedure's body and the end of any nested definitions

If procedure definitions are nested, so are the *procedure endproc* brackets. Variable and type definitions made at the outermost level appear outside of all procedure/endproc items.

Information about the relationship between the executable code and source files is collected together and appears as a *fileinfo* item, which is always the final item in a debugging area. Because of the C language's #include facility, the executable code produced from an outer-level source file may be separated into disjoint pieces interspersed with that produced from the included files. Therefore, source files are considered to be collections of *fragments*, each corresponding to a contiguous area of executable code, and the *fileinfo* item is a list with an entry for each file, each in turn containing a list with an entry for each fragment. The *fileinfo* field in the *section* item addresses the *fileinfo* item itself. In each *procedure* item there is a *fileentry* field, which refers to the file-list entry for the source file containing the procedure's start; there is a separate one in the *endproc* item because it may possibly not be in the same source file.

15.2.1 Endianness and the encoding of debugging data

The ARM can be configured to use either a little-endian memory system (least significant byte of each four-byte word has the lowest address), or a big-endian memory system (most significant byte of each four-byte word has the lowest address).

In general, the code to be generated varies according to the byte sex (or endianness) of the target, and the linker has insufficient information to change the byte sex of an object file. Therefore, object files are encoded using the byte order of the intended target, independently of the byte order of the host system on which the compiler or assembler runs. The ARM linker accepts inputs having either byte order, but rejects mixed sex inputs, and generates its output using the same byte order. This means that producers of debugging tables must be prepared to generate them in either byte order, as required. In turn, this requires definitions to be very clear about when a four-byte word is being used (which will

require reversal on output or input when cross-sex compiling or debugging), and when a sequence of bytes is being used (which requires no special treatment provided it is written and read as a sequence of bytes in address order).

15.3 Representation of Data Types

Several of the debugging data items (eg. procedure and variable) have a *type* word field to identify their data type. This field contains:

- in the most significant 24 bits, a code to identify a base type
- in the least significant 8 bits, a pointer count:
 - 0 denotes the type itself
 - 1 denotes a pointer to the type
 - 2 denotes a pointer to a pointer

For simple types the code is a positive integer as follows (all codes are decimal):

void	0
signed integers	
single byte	10
halfword	11
word	12
double word	13
unsigned integers	
single byte	20
halfword	21
word	22
double word	23
floating point	
float	30
double	31
long double	32
complex	
single complex	41
double complex	42
functions	
function	100

For compound types (arrays, structures, etc.) there is a special kind of debug data item (array, struct, etc.) to give details such as array bounds and field types. The type code for compound types is negative; the negation of the (byte) offset of the debug item from the start of the debugging area.

Set types in Pascal are not treated in detail: the only information recorded for them is the total size occupied by the object in bytes. Neither are Pascal *file* variables supported by the debugger, since their behavior under debugger control is unlikely to be helpful to you.

Fortran character types are supported by special kinds of debugging data item, the format of which is specific to each Fortran compiler.



15.3.1 Representation of source file positions

Several of the debugging data items have a *sourcepos* field to identify a position in the source file. This field contains a line number and character position within the line packed into a single word. The most significant 10 bits encode the character offset (zero-based) from the start of the line and the least significant 22 bits give the line number.

15.3.2 The code and length field

The first word of each debugging data item contains the byte length of the item (encoded in the most significant 16 bits), and a code identifying the kind of item (in the least significant 16 bits). The defined codes are:

1	section
2	procedure/function definition
3	endproc
4	variable
5	type
6	struct
7	array
8	subrange
9	set
10	fileinfo
11	contiguous enumeration
12	discontiguous enumeration
13	procedure/function declaration
14	begin naming scope
15	end naming scope
16	bitfield
17	macro definition
18	macro undefinition
19	class
20	union
32	FP map fragment

The meaning of the second and subsequent words of each item is defined in the following sections.

If a debugger encounters a code it does not recognize, it uses the length field to skip the item entirely. This discipline allows the debugging tables to be extended without invalidating existing debuggers.

ARM Symbolic Debug Table Format

15.3.3 Text names in items

Where items include a string field, the string is packed into successive bytes beginning with a length byte, and padded at the end to a word boundary with 0 bytes. The length of a string is in the range [0..255] bytes.

15.3.4 Offsets in file and addresses in memory

When an item contains a field giving an offset in the debugging data area (usually to address another item), this means a byte offset from the start of the debugging data for the whole section (in other words, from the start of the *section* item).



15.4 Section Items

A section item is the first item of each section of the debugging data. After its code and length word it contains the fields listed below. First, there are four flag bytes:

lang	a byte identifying the source language
flags	a byte describing the level of detail
unused	
asdversion	a byte version number of the debugging data

15.4.1 Lang byte

The language byte codes are defined in the following table:

Language	Code	Description
LANG_NONE	0	Low-level debugging data only
LANG_C	1	C source level debugging data
LANG_PASCAL	2	Pascal source level debugging data
LANG_FORTRAN	3	Fortran-77 source level debugging data
LANG_ASM	4	ARM assembler line number data

Table 15-1: Language byte codes

All other codes are reserved by ARM.

15.4.2 Flags byte

The flags byte uses the following mask values:

- debugging data contains line-number information.
- debugging data contains information about top-level variables.
- both of the above

15.4.3 asdversion byte

The asdversion byte should be set to 2, the version of this definition.

The flag bytes are followed by the following word-sized fields:

codestart	Address of first instruction in this section, relocated by the linker.
datastart	Address of start of static data for this section, relocated by the linker.
codesize	Byte size of executable code in this section.

ARM Symbolic Debug Table Format

datasize	Byte size of the static data in this section.																				
fileinfo	Offset in the debugging area of the fileinfo item for this section (0 if no fileinfo item present). The fileinfo field is 0 if no source file information is present.																				
debugsize	Total byte length of debug data for this section.																				
name or nsyms	String or integer. The name field contains the program name for Pascal and Fortran programs. For C programs it contains a name derived by the compiler from the root filename (notionally a module name). In each case, the name is similar to a variable name in the source language. For a low-level debugging section (language = 0), the field is treated as a four-byte integer giving the number of symbols following. For linker-generated debug data, the fields have these values: <table><tr><td>language</td><td>0</td></tr><tr><td>codestart</td><td>Image\$\$RO\$\$Base</td></tr><tr><td>datastart</td><td>Image\$\$RW\$\$Base</td></tr><tr><td>codesize</td><td>Image\$\$RO\$\$Limit - Image\$\$RO\$\$Base</td></tr><tr><td>datasize</td><td>Image\$\$RW\$\$Limit - Image\$\$RW\$\$Base</td></tr><tr><td>fileinfo</td><td>0</td></tr><tr><td>nsyms</td><td>Number of symbols in the next debugging data.</td></tr><tr><td>debugsize</td><td>Total size of the low-level debugging data including the size of this section item.</td></tr></table> Also, the section item is followed by nsyms symbol items, each consisting of two words: <table><tr><td>sym</td><td>Flags + byte offset in string table of symbol name. sym encodes an index into the string table in the 24 least significant bits, and the following flag values in the eight most significant bits, as in Table 15-2: Linker-generated debugging data.</td></tr><tr><td>value</td><td>the symbol's value.</td></tr></table>	language	0	codestart	Image\$\$RO\$\$Base	datastart	Image\$\$RW\$\$Base	codesize	Image\$\$RO\$\$Limit - Image\$\$RO\$\$Base	datasize	Image\$\$RW\$\$Limit - Image\$\$RW\$\$Base	fileinfo	0	nsyms	Number of symbols in the next debugging data.	debugsize	Total size of the low-level debugging data including the size of this section item.	sym	Flags + byte offset in string table of symbol name. sym encodes an index into the string table in the 24 least significant bits, and the following flag values in the eight most significant bits, as in Table 15-2: Linker-generated debugging data .	value	the symbol's value.
language	0																				
codestart	Image\$\$RO\$\$Base																				
datastart	Image\$\$RW\$\$Base																				
codesize	Image\$\$RO\$\$Limit - Image\$\$RO\$\$Base																				
datasize	Image\$\$RW\$\$Limit - Image\$\$RW\$\$Base																				
fileinfo	0																				
nsyms	Number of symbols in the next debugging data.																				
debugsize	Total size of the low-level debugging data including the size of this section item.																				
sym	Flags + byte offset in string table of symbol name. sym encodes an index into the string table in the 24 least significant bits, and the following flag values in the eight most significant bits, as in Table 15-2: Linker-generated debugging data .																				
value	the symbol's value.																				

Symbol	Offset	Description
ASD_ABSSYM	0	if the symbol is absolute
ASD_GLOBSYM	0x01000000L	if the symbol is global
ASD_TEXTSYM	0x02000000L	if the symbol names code
ASD_DATASYM	0x04000000L	if the symbol names data
ASD_ZINITSYM	0x06000000L	if the symbol names 0-initialized data
ASD_16BITSYM	0x10000000L	bit set if the symbol is a Thumb symbol

Table 15-2: Linker-generated debugging data



Note *The linker reduces all symbol values to absolute values, so that the flag values record the history, or origin, of the symbol in the image.*

Immediately after the symbol table is the string table, in standard AOF format. It consists of:

- a length word
- the strings themselves, each terminated by a NUL (0)

The length word includes the size of the length word, so no offset into the string table is less than four. The end of the string table is padded with NULs to the next word boundary (so the length is a multiple of 4).

15.5 Procedure Items

A procedure item appears once for each procedure or function definition in the source program. Any definitions within the procedure have their related debugging data items between the procedure item and its matching endproc item. After its code and length field, a procedure item contains the following word-sized fields:

type	the return type if this is a function, else 0 (see 15.3 Representation of Data Types on page 15-4)
args	the number of arguments
sourcepos	the source position of the procedure's start (see 15.3.1 Representation of source file positions on page 15-5)
startaddr	address of the first instruction of the procedure prologue The <code>startaddr</code> field addresses the start of the prologue; it is the instruction where control arrives when the procedure is called.
entry	address of the first instruction of the procedure body The <code>entry</code> field addresses the first instruction following the procedure prologue; it is the first address where a high-level breakpoint could sensibly be set.
endproc	offset of the related endproc item
fileentry	offset of the file list entry for the source file
name	string

15.5.1 Type items

A type item is used to describe a named type in the source language (eg. a typedef in C). After its code and length field, a type item contains two word-sized fields:

type	a type word (described in 15.3 Representation of Data Types on page 15-4)
name	string

15.5.2 Endproc items

An endproc item marks the end of the debugging data items belonging to a particular procedure. It also contains information relating to the procedure's return. After its code and length field, an endproc item contains the following word-sized fields:

sourcepos	position in the source file of the procedure's end (see 15.3.1 Representation of source file positions on page 15-5)
endpoint	address of the code byte <i>after</i> the compiled code for the procedure
fileentry	offset of the file-list entry for the procedure's end
nreturns	number of procedure return points (may be 0)
retaddr	array of addresses of procedure return code

If the procedure body is an infinite loop, there will be no return point, so `nreturns` will be 0. Otherwise each member of `retaddr` should point to a suitable location at which a breakpoint may be set *at the exit of the procedure*. When execution reaches this point, the current stack frame should still be for this procedure.

15.5.3 Label items

A label in a source program is represented by a special procedure item with no matching endproc (the endproc field is 0 to denote this). Pascal and Fortran numerical labels are converted by their respective compilers into strings prefixed by \$n. For Fortran77, multiple entry points to the same procedure each give rise to a separate procedure item, all of which have the same endproc offset referring to the unique, matching endproc item.

15.5.4 Variable items

A variable item contains debugging data relating to a source program variable, or a formal argument to a procedure (the first variable items in a procedure always describe its arguments). After its code and length field, a variable item contains the following word-sized fields:

type	type of this variable (see 15.3 Representation of Data Types on page 15-4)
sourcepos	the source position of the variable (see 15.3.1 Representation of source file positions on page 15-5)
storageclass	a word encoding the variable's storage class
location	see explanation below
name	string



The following codes define the storage classes of variables:

- 1 external variables (or Fortran common)
- 2 static variables private to one section
- 3 automatic variables
- 4 register variables
- 5 Pascal 'var' arguments
- 6 Fortran arguments
- 7 Fortran character arguments

The meaning of the location field of a variable item depends on the storage class:

- an absolute address for static and external variables (relocated by the linker)
- a stack offset (offset from the frame pointer) for automatic and var-type args
- an offset into the argument list for Fortran argument
- a register number for register variables (the eight floating-point registers are 16..23)

The `sourcepos` field is used by the debugger to distinguish between different definitions that have the same name (eg. identically named variables in disjoint source-level naming scopes such as nested blocks in C).

15.5.5 Struct, union, and class items

A struct item is used to describe a structured data type (eg. a struct in C or a record in Pascal). A class item is used to describe a C++ class type. A union item is used to describe a union type. All have the same format. Note that the C or C++ tag for a struct, union, or class is not represented in the debug table.

After its code and length field, a struct item contains the following word-sized fields:

<code>fields</code>	the number of fields in the structure
<code>size</code>	total byte size of the structure
<code>fieldtable...</code>	an array of <code>fields</code> struct field items

Each struct field item has the following word-sized fields:

<code>offset</code>	byte offset of this field within the structure
<code>type</code>	a type word (described in 15.3 Representation of Data Types on page 15-4)
<code>name</code>	string

Earlier versions of the ARM tools described union types by struct items in which all fields have 0 offsets.

For C and C++ bit fields, the type part of the type word identifies a bitfield item.

15.5.6 Array items

An array item is used to describe a one-dimensional array. Multi-dimensional arrays are described as *arrays of arrays*. Which dimension comes first is dependent on the source language (which is different for C and Fortran). After its code and length field, an array item contains the following word-sized fields:

size	total byte size of the array. If the size field is zero, debugger operations which affect the whole array, rather than individual elements of it, are forbidden.
flags	(see below)
basetype	a type word (described in 15.3 Representation of Data Types on page 15-4)
lowerbound	constant value or location of variable
upperbound	constant value or location of variable

The following mask values are defined for the `flags` field:

ARRAY_UNDEF_LBOUND	1	lower bound is undefined
ARRAY_CONST_LBOUND	2	lower bound is a constant
ARRAY_UNDEF_UBOUND	4	upper bound is undefined
ARRAY_CONST_UBOUND	8	upper bound is a constant
ARRAY_VAR_LBOUND	16	lower bound is a variable
ARRAY_VAR_UBOUND	32	upper bound is a variable

15.5.7 Bounds

Note *A variable item may be used to describe a location known to the compiler, which need not correspond to a source language variable.*

undefined	No information about it is available.
constant	Its value is known at compile time. In this case, the corresponding bound field gives its value.
variable	The offset field identifies a variable debug item describing the location containing the bound. In a debug area in an object file, the offset field contains the offset from the start of the debug area to the variable item.



15.5.8 Subrange items

A subrange item is used to describe a subrange typed in Pascal. It also serves to describe enumerated types in C, and scalars in Pascal (in which case the base type is understood to be an unsigned integer of appropriate size). After its code and length field, a subrange item contains the following word-sized fields:

lb	low bound of subrange
hb	high bound of subrange
sizeandtype	encodes the byte size of container for the subrange (1, 2 or 4) in its least significant 16 bits, and a simple type code (see 15.3 Representation of Data Types on page 15-4) in its most significant 16 bits. The type code refers to the base type of the subrange; for example, a subrange 256..511 of unsigned short may be held in one byte.

15.5.9 Set items

A set item is used to describe a Pascal set type. Currently, the description is only partial. After its code and length field, a set item consists of a single word:

size	byte size of the object
------	-------------------------

15.5.10 Enumeration items

An enumeration item describes a Pascal or C enumerated type. After its code and length word, the description of a *contiguous enumeration* contains the following word-sized fields:

type	a type word describing the type of the container for the enumeration (see 15.3 Representation of Data Types on page 15-4)
count	the cardinality of the enumeration
base	the first (lowest) value (may be negative)
nametable	a character array containing <i>count</i> name strings

The description of a *discontiguous enumeration* (such as the C enumeration enum bits {bit0=1, bit1=2, bit2=4, bit3=8, bit4=16}) contains the following fields after its code and length word:

type	as above
count	as above
nametable	a table of <i>count</i> (value, name) pairs

Each nametable entry has the following format (which is variable in length):

val	the enumerated value (1/2/4/8/16 in the example)
name	string (may be several words long)

15.5.11 Function declaration items

After its code and length word, a function declaration item contains the following fields:

type	a type word (see 15.3 Representation of Data Types on page 15-4) describing the return type of the function or procedure
argcount	the number of arguments to the function
args	a sequence of <i>argcount</i> argument description items

Each argument description item contains the following:

type	a type word (see 15.3 Representation of Data Types on page 15-4) describing the type of the argument
name	string (may be several words)

An argument descriptor need not be named; in this case the length of the name is zero, and the name field is a single zero word.

15.5.12 Begin and end naming scope items

These debug items are used to mark the beginning and end of a naming scope. They must be properly nested in the debug area. In each case, after the code and length word, there is one word-sized field:

codeaddress	address of the start/end of scope (which is determined by the code word)
-------------	--

15.5.13 Bitfield item

A bitfield item describes a bitfield member of a C or C++ struct, union, or class. After the code and length field, a bitfield item contains the following fields, which are then followed by two zero bytes to pad to a word boundary:

type	a type word describing the type of the field
container	a type word describing the type of the field's container
size	a byte containing the size of the field in bits
offset	a byte containing the offset from bit 0 of the containing value of bit 0 of the field

15.5.14 Macro definition item

A macro definition item describes a C or C++ preprocessor macro definition (`#define`). After the code and length field, a macro definition item contains the following fields:

fileentry	offset of the file list entry for the source file containing the macro definition
sourcepos	the source position of the macro definition
body	the offset of the replacement for the macro (not a string, but the characters of the replacement, terminated by a zero byte)



ARM Symbolic Debug Table Format

argcount	a word containing the number of arguments for the macro. For object-type macros, this field has the value -1.
argtable	offset of a description of the macro's argument names. This contains argcount strings. The field is zero if the macro has no arguments.
name	string

Note *The body and argtable offsets are contained within the macro definition item, but the offset is an offset within the containing section.*

15.5.15 Macro undefinition item

A macro undefinition item describes a C or C++ preprocessor macro undefinition (#undef). After the code and length field, a macro undefinition item contains the following fields:

fileentry	offset of the file list entry for the source file containing the macro undefinition
sourcepos	the source position of the macro undefinition
name	string

15.5.16 Fileinfo items

A fileinfo item appears once per section, after all other debugging data items. If the fileinfo item is too large for its length to be encoded in 16 bits, its length field must be written as 0 (since this is the last item in a section and the section header contains the length of the whole section, the length field is strictly redundant).

Each source file is described by a sequence of *fragments*. Each *fragment* describes a contiguous region of the file, within which the addresses of compiled code increase monotonically with source file position. The order in which fragments appear in the sequence is not necessarily related to the source file positions to which they refer.

Note *For compilations which make no use of the #include facility, the list of fragments may have only one entry, and all line-number information can be contiguous.*

After its code and length word, the fileinfo item is a sequence of file entry items of this format:

len	length of this entry in bytes (including the length of the following fragments)
date	date and time when the file was last modified (may be 0, indicating not available, or unused). If present, the date field contains the number of seconds since the beginning of 1970 (the UNIX date origin).
filename	string (or "" if the name is not known)
fragment data	see below

Following the final file entry item, a single 0 word marks the end of the sequence.

The fragment data is a word giving the number of following fragments, followed by a sequence of fragment items:



ARM Symbolic Debug Table Format

n	number of fragments following
fragments...	n fragment items
Each fragment item consists of five words, followed by a sequence of byte pairs and halfword pairs, formatted as follows:	
size	length of this fragment in bytes (including length of following lineinfo items)
firstline	linenumber
lastline	linenumber
codestart	pointer to the start of the fragment's executable code
codesize	byte size of the code in the fragment
lineinfo...	a variable number of bytes matching line numbers to code addresses

Each lineinfo item describes a source statement and consists of a pair of (unsigned) bytes, possibly followed by two or three (unsigned) halfwords (each halfword has the byte ordering appropriate to the target memory system's endianness or byte sex).

The short form (pair of bytes) lineinfo item is as follows:

codeinc	# bytes of code generated by this statement. If codeinc is greater than 255, or lineinc is required to describe a line number change greater than 63 or a column change greater than 191, then both bytes are written to describe 0 increments, and the real values are given in the following two or three (unsigned) halfwords.
lineinc	# source space occupied by this statement. lineinc describes how to calculate the source position (line, column) of the next statement from the source position of this one. If lineinc is in the range 0 <= lineinc < 64, the new position is (line+lineinc,1). If lineinc >= 64, the new position is (line,column+lineinc-64).

The number of bytes of code generated for a statement may be zero, provided that the line increment is non-zero (such an item may describe a block end or block start).

It is not possible to describe a statement which generates no code and no line number increment, as encoding is used as an escape to the long form lineinfo items.

Note *There are two ways to describe zero increments: zero lines and zero columns, which serves to discriminate between the two halfword and three halfword forms.*

If the starting column for the next statement is 1, the two-halfword form is used, which in effect is a triple of halfwords as follows:

zero	2 zero bytes
lineinc	# source lines occupied by this statement
codeinc	# bytes of code generated by this statement

Note *The order of the lineinc and codeinc halfwords is the reverse of the corresponding bytes.*



ARM Symbolic Debug Table Format

If the starting column for the next statement is not 1, the three-halfword form is used, which in effect is a quadruple of halfwords, as follows:

```
codeinc = 0, lineinc = 64
lineinc      # source lines occupied by this statement
codeinc      # bytes of code generated by this statement
newcol       starting column for the next statement
```

Note *Again, the order of the lineinc and codeinc halfwords is the reverse of the corresponding bytes. In addition, the column item here is the absolute column number for the next statement, and not an increment as in the two byte form.
(This encoding of lineinfo items is an incompatible change from the previous format (version 2): in that format, lineinc in a two byte lineinfo item always describes a line increment, and accordingly, there is no four halfword form. Programs interpreting asd tables should interpret lineinfo items differently according to the table format in the section item.)*

15.5.17 Map fragment items

An FP map fragment item describes the offsets from the stack pointer of the “virtual frame pointer” in functions without a frame pointer. The stack offsets in variable items are offsets from its virtual frame pointer. Functions may have no frame pointer because a no-fp APCS variant has been selected, or because they are sufficiently simple leaf functions with an fp APCS variant. In the latter case, FP map fragments will be generated only for those functions which actually use the stack.

FP map fragments are generated regardless of whether other debug tables are being generated, in a separate debug area (whether or not the -g compiler option is used).

After the code and length field, an FP map fragment item contains the following fields:

bytes	exact size of the item in bytes (the length part of the code and length field is rounded up to a word boundary)
codestart	address of the first word of code described
saveaddr	address of the instruction saving callee-save registers. (0 if there is none within the area of code described)
codesize	size of the area of code described
initoffset	offset of the virtual FP from the SP at the start of the code area described. These items are followed by a variable number of FPInfo fields, which take one of these forms:

byte codeinc	size of code area described by this item (unsigned)
--------------	--

byte offsetinc	change in the virtual FP offset for the next item (signed)
----------------	---

or, two zero bytes:

short codeinc	(unsigned)
---------------	------------

short offsetinc	(signed)
-----------------	----------

